

TEST DESIGN AND EXECUTION

Objectives of Testing

Delivering quality products is the ultimate objective of testing. The various objectives of testing:

- Identification of Bugs and Errors
- Delivering Quality Product
- Justification with Requirement
- Increasing Confidence in the Product
- Enhanced Growth

1. Identification of Bugs and Errors

Once the developer finishes coding, the tester starts testing. During testing, QA validates each module under various conditions. Afterward, they collect all the errors & bugs and forward them to the developer to fix. With modern advancements like AI/ML for test case generation, better analysis, and execution, the testing processes are now capable of delivering better products. Other methods include Shift-Left Testing and Robotic Process Automation (RPA) Testing which helps in maximizing test coverage.

2. Delivering Quality Product

The main aim of testing is to maintain the quality of the product. Also, testing has its cycle and in each phase, all focus revolves around quality only. These include using the core testing methods to perform functional testing and non-functional testing. The emergence of DevOps combined with Quality Assurance (QA) to utilize QAOps along with Agile methodologies has certainly boosted the overall productivity to deliver quality products.

3. Justification with Requirement

During testing, the QA team validates whether the application is following the SRS (System Requirement Specification) document or not. Testing for user experience and implementing API testing helps in validating the business requirements of the product.

4. Increasing Confidence in the Product

The testing team constantly checks the features of the software. It must fulfill the business demand and drive confidence. [Security, performance](#), and scalability testing enabled through automation with CI/CD pipelines through DevOps helps in building and increasing confidence. This assures the reliability of the product.

5. Enhanced Growth

A quality delivery increases the potential of a business. And we all know quality comes through testing only. Increased ROI through proper testing helps in the sustained and continuous growth of the product. Business and user goals are properly met with higher revenues and improved cost efficiency.

TEST DESIGN FACTORS

Test design factors in software testing encompass the key elements and considerations that guide the creation of effective test cases and ensure comprehensive testing of a software application. These factors include test objectives, scenarios, data, techniques, and the overall test plan, all working together to verify functionality, usability, security, and other aspects of the software.

Key Factors of Test Design:

➤ Test Objectives:

Defining the specific goals of the testing effort, such as verifying system functionality, ensuring user-

friendliness, and assessing security vulnerabilities.

➤ Test Scenarios:

Outlining the different paths a user might take through the application, covering various user interactions and potential scenarios.

➤ Test Data:

Selecting the appropriate data inputs for test execution, including valid, invalid, and boundary value data.

➤ Test Design Techniques:

Employing various methods to generate test cases, such as black-box, white-box, and grey-box techniques.

➤ Test Plan:

Creating a comprehensive document that outlines the testing approach, scope, and success criteria.

Types of Test Design Techniques:

Black-box testing:

Focuses on the functionality of the software without considering its internal structure or code.

- Equivalence Partitioning: Dividing the input data into classes based on their expected behavior.
- Boundary Value Analysis: Testing the boundaries of input and output values to identify potential errors.
- Decision Table Testing: Using a table to represent the relationships between conditions and actions.

White-box testing:

Focuses on the internal structure and code of the software.

- Statement Coverage: Ensuring that all statements in the code are executed during testing.
- Branch Coverage: Ensuring that all branches in the code are executed during testing.

Grey-box testing:

Combines elements of both black-box and white-box testing.

Other Techniques:

- State Transition Testing: Testing the transitions between different states in the software.
- Use Case Testing: Testing the application based on user stories and scenarios.
- Error Guessing: Intentionally introducing errors to identify potential vulnerabilities.

Test Environment:

- A crucial factor in test design is setting up a test environment that mimics the production environment as closely as possible.
- This includes hardware, software, network configurations, and other dependencies needed for accurate test execution.

Test Automation:

- Automating tests can save time and effort, especially for repetitive or complex tests.
- It ensures consistent execution and can be used to identify regressions.

Test Data:

- Test data is essential for ensuring the functionality and performance of the software.
- It should be representative of the range of inputs and conditions the software will encounter in real-world use.

Risk-Based Testing:

- Prioritizing tests based on the potential risk of failure and the impact on the software's functionality.
- This helps to focus testing efforts on the most critical areas.

Continuous Testing:

- Integrating testing throughout the software development lifecycle, from early stages to deployment.
- This helps to catch defects early and improve software quality.

REQUIREMENT IDENTIFICATION

Requirement identification in software testing is the process of clearly defining and documenting the needs and functionalities of a software product to ensure it meets user expectations and business goals. It involves identifying stakeholders, gathering requirements, categorizing them, and ensuring they are complete, consistent, and testable. This process is crucial for effective testing and helps to minimize errors and development costs.

Steps in Requirement Identification:

1. Identify Key Stakeholders and End Users:

Determine who will use the software and their respective roles and needs.

2. Understand the Project Goal:

Clearly define what the software is intended to achieve and its overall objectives.

3. Gather Requirements:

Collect information about user needs, functional requirements, and non-functional requirements (e.g., performance, security).

4. Categorize and Define Requirements:

Organize the gathered requirements into logical categories and provide detailed descriptions for each.

5. Analyze Requirements:

Evaluate the requirements for completeness, consistency, and testability.

6. Model Requirements:

Use diagrams or other visual representations to illustrate the software's functionality and interactions.

7. Document and Review Requirements:

Create a Software Requirements Specification (SRS) document to formally document the requirements and ensure all stakeholders agree on them.

8. Finalize SRS:

Obtain sign-off from stakeholders on the SRS to confirm its completeness and accuracy.

Benefits of Effective Requirement Identification:

➤ Reduced Development Costs:

Early identification and documentation of requirements helps to avoid costly rework later in the development process.

➤ Improved Software Quality:

By ensuring the software meets user needs and business goals, it leads to higher quality and better user satisfaction.

➤ Effective Testing:

Well-defined requirements provide a clear basis for developing test cases and ensuring thorough testing.

➤ Enhanced Communication:

A well-documented SRS facilitates communication between stakeholders, developers, and testers, leading to better alignment and collaboration.

TESTABLE REQUIREMENTS

Testable requirements in software testing are precise and unambiguous descriptions of a system's functionality, allowing for the creation of test cases to verify that the requirements are met. They are clear, measurable, and complete, leaving no room for interpretation and ensuring that test cases can effectively demonstrate whether a requirement is fulfilled.

Characteristics of Testable Requirements:

➤ Clarity:

Requirements should be written in a way that is easy to understand and leaves no room for ambiguity.

➤ Measurability:

Requirements should be expressed in a way that allows for objective assessment, using quantifiable or observable indicators.

➤ **Completeness:**

Requirements should cover all aspects of the functionality being tested, leaving no gaps or assumptions.

➤ **Specificity:**

Requirements should be specific and avoid general terms or vague language.

➤ **Achievability:**

Requirements should be realistic and achievable within the constraints of the project.

➤ **Relevance:**

Requirements should be relevant to the project and the needs of the stakeholders.

➤ **Traceability:**

Requirements should be traceable back to the business needs and user stories, ensuring that the development process is aligned with the overall goals.

Benefits of Testable Requirements:

➤ **Improved Test Case Design:**

Well-defined testable requirements make it easier to design effective test cases that cover all aspects of the functionality being tested.

➤ **Reduced Misunderstandings:**

Clear and unambiguous requirements reduce the risk of misunderstandings between developers, testers, and stakeholders, leading to fewer defects and rework.

➤ **Early Defect Detection:**

Testable requirements facilitate early detection of defects, which can save time and cost by preventing them from propagating through the development process.

➤ **Increased Confidence in the Software:**

Thoroughly tested software built on testable requirements provides stakeholders with increased confidence in the quality and reliability of the system.

➤ **Examples of Testable Requirements:**

"The login form must display an error message when the user enters an incorrect username or password".

"The system must save user data to the database within 2 seconds".

"The user interface must be responsive to different screen sizes".

How to Write Testable Requirements?

➤ **Break down complex requirements into smaller, more manageable chunks:**

This makes it easier to define clear and measurable criteria for each individual requirement.

➤ **Use a formal requirements specification document:**

This ensures that all requirements are documented in a structured and organized way.

➤ **Use the SMART acronym (Specific, Measurable, Achievable, Relevant, Time-bound) to guide the writing process:**

This helps ensure that requirements are clear, measurable, and achievable.

➤ **Involve stakeholders in the requirements gathering process:**

This ensures that the requirements are aligned with the needs of the stakeholders and that there is a shared understanding of what is required.

➤ **Review and refine requirements iteratively:**

This ensures that the requirements are accurate, complete, and testable throughout the development process.

TEST DESIGN PROCESS

Test design in software testing is the process of creating a strategic plan for identifying test cases, scenarios, and conditions to ensure the software or system meets its requirements and behaves as

expected under various conditions. It involves understanding requirements, identifying test conditions, and designing test cases, selecting or creating test data, developing test scripts if needed, and reviewing and refining the design to ensure comprehensive coverage.

Here's a more detailed breakdown of the test design process:

1. Understanding Requirements:

Thoroughly analyze the software requirements, specifications, and functional descriptions to understand what needs to be tested.

Identify the key functionalities, features, and business rules that need to be validated.

2. Identifying Test Conditions:

Based on the requirements, define the specific conditions under which the software should be tested.

These conditions should cover various scenarios, including normal, edge, and error cases.

3. Designing Test Cases:

Develop test cases that cover the identified test conditions.

Utilize test case design techniques like equivalence partitioning, boundary value analysis, decision table testing, state transition testing, and use case testing.

Test cases should be clear, concise, and well-structured, with clearly defined steps, expected results, and pass/fail criteria.

4. Selecting or Creating Test Data:

Choose or create appropriate test data that will be used to execute the test cases.

Test data should represent a variety of inputs, including valid and invalid data, as well as data that may cause errors.

5. Developing Test Scripts (if needed):

If automating the test execution, develop test scripts using automation testing tools and frameworks.

Test scripts should be well-structured and maintainable, allowing for easy modification and reuse.

TEST RESULTS

Test results in software testing are the outcomes of executing test cases, revealing whether the software behaves as expected. These results are crucial for understanding the quality of the software and informing decisions about its readiness for release.

Key aspects of test results:

- Expected vs. Actual Results:

Test results involve comparing the expected outcome of a test case with the actual outcome observed when the test is executed.

- Passing and Failing Tests:

Tests are categorized as passing or failing based on whether the actual result matches the expected result.

- Defect Reporting:

Failing tests often indicate defects in the software, which are then reported and tracked for resolution.

- Test Reports:

Test results are often documented in test reports, which summarize the testing process, results, and any identified defects.

- Decision-Making:

Test results help stakeholders make informed decisions about the software's quality, release readiness, and whether further testing or development is needed.

Importance of Test Results:

- Quality Assurance:

Test results ensure the software meets the defined requirements and functions correctly, contributing to its overall quality.

- Defect Management:

Identifying and reporting defects early in the testing process allows for timely resolution, reducing

the cost and effort required later in development.

- **Release Decisions:**

Test results provide the information needed to determine if the software is ready for release, ensuring that only high-quality software is deployed to users.

- **Stakeholder Communication:**

Test reports provide a clear and concise summary of the testing process and results, enabling stakeholders to understand the status of the software and make informed decisions.

- **Continuous Improvement:**

Analyzing test results can help identify areas for improvement in the software development process, leading to better software quality in future projects.

What is Test Reporting in Software Testing?

Test reporting in software testing is the process of documenting a detailed report that summarizes the testing process, its results, objectives, and the effectiveness of the testing. This report includes details about any defects found, how the software performed, and suggests areas for improvement.

Steps in the Test Reporting Phase

- Collecting and analyzing test results.
- Preparing comprehensive test reports.
- Reviewing test reports and identifying defects.
- Prioritizing and assigning defects to developers for resolution.
- Verifying defect fixes.
- Re-testing defects.
- Tracking and reporting progress on defect resolution.
- Analyzing and reporting on overall test results.
- Preparing final test reports for stakeholders.

Test Design Preparedness Metrics

- Metrics are tracked
 - To know if a test project is progressing according to schedule
 - Plan the next project more accurately
- The following metrics are monitored
 - Preparation status of test cases (PST)
 - Average time spent (ATS) in test case design
 - Number of available test (NAT) cases
 - Number of planned test (NPT) cases
 - Coverage of a test suite (CTS)

Test Case Design Effectiveness

- The objective of the test design effectiveness metric is to
 - Measure the *defect revealing ability* of the test suite
 - Use the metric to improve the test design process
- A metric commonly used in the industry to measure test case design effectiveness is the Test Case Design Yield (TCDY)

$$TDCY = \frac{NPT}{NPT + \text{Number of TCE}} \times 100\%$$

- TCE stand for Test Case Escaped
- During testing new defects are found for which no test cases had been planned
- For these new defects, new test cases are designed, which is known as Test Case Escaped metric

MODEL-DRIVEN TEST DESIGN

The methodology is also known as model-driven software development (MDSD), model-driven engineering (MDE) and model-driven architecture (MDA). The MDD approach focuses on the construction of a software model. The model is a diagram that specifies how the software system should work before the code is generated.

What is Model-Driven Test Design?

MDTD is built on the idea that designers will become more effective and efficient if they can raise the level of abstraction. This approach breaks down the testing into a series of small tasks that simplify test generation. Then test designers isolate their tasks and work at a higher level of abstraction by using mathematical engineering structures to design test values independently of the details of the software or design artifacts, test automation, and Test Execution.

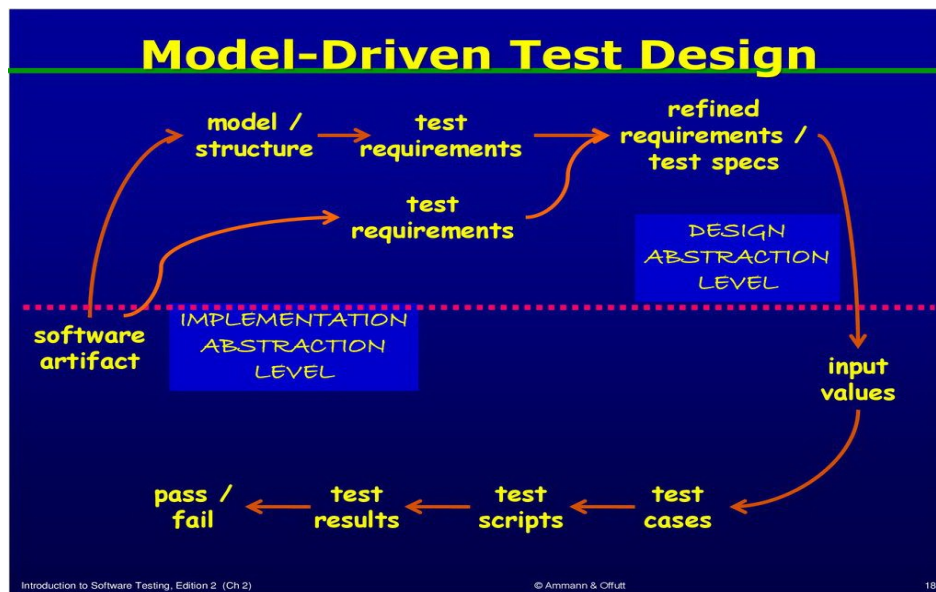
Different phases in MDTD

MDTD can be done in 4 different phases. Each type of activity requires different skills, background knowledge, education, and training. It is better to use different sets of people depend on the situation. 1. Test Design — This can be done in either Criteria-Based where Design test values satisfy coverage criteria or other engineering goals or in Human-Based where Design test values based on domain knowledge of the program and human knowledge of testing which is comparatively harder. This the most technical part of the MDTD process better to use experienced developers in this phase.

2. Test Automation — This involves embedding test values to scripts. Test cases are defined based on the test requirements. Test values are chosen such that we can cover a larger part of the application with fewer test cases. We don't need that much domain knowledge in this phase, however, we need to use technically skilled people.

3. Test Execution — The test engineer will run tests and records the results in this activity. Unlike the previous activities, test execution not required a high skill set such as technical knowledge, logical thinking, or domain knowledge. Since we consider this phase comparatively low risk, we can assign junior intern engineers to execute the process. But we should focus on monitoring, log collecting activities based on automation tools.

4. Test Evaluation — The process of evaluating the results and reporting to developers. This phase is comparatively harder and we expected to have knowledge in the domain, testing, and User interfaces, and psychology



The starting point in Figure 2.4 is a software artifact. This could be program source, a UML diagram, natural language requirements, or even a user manual. A criteria-based test designer uses that artifact to create an abstract model of the software in the form of an input domain, a graph, logic expressions, or a syntax description. Then a coverage criterion is applied to create test requirements. A human-based test designer uses the artifact to consider likely problems in the software, then creates requirements to test for those problems. These requirements are sometimes refined into a more specific form, called the test specification. For example, if edge coverage is being used, a test requirement specifies which edge in a graph must be covered. □

TEST CASE ORGANIZATION AND TRACKING

A **Test Case** is a set of actions executed to verify a particular feature or functionality of your software application. A Test Case contains test steps, test data, precondition, postcondition developed for specific test scenario to verify any requirement. The test case includes specific variables or conditions, using which a testing engineer can compare expected and actual results to determine whether a software product is functioning as per the requirements of the customer.

Test Scenario Vs Test Case

Test scenarios are rather vague and cover a wide range of possibilities. Testing is all about being very specific.

For a Test Scenario: Check Login Functionality there many possible test cases are:

- Test Case 1: Check results on entering valid User Id & Password
- Test Case 2: Check results on entering Invalid User ID & Password
- Test Case 3: Check response when a User ID is Empty & Login Button is pressed, and many more

BUG REPORTING

Bug reporting in software testing is a structured process of documenting and communicating issues found in software. It's a crucial part of ensuring the final product meets quality standards and functions as intended. Effective bug reports provide developers with the necessary information to understand, reproduce, and fix issues, ultimately improving software reliability and stability.

Here's a more detailed look at bug reporting in software testing:

Key Aspects of Bug Reporting:

- Documentation:
Bug reports are written documents that detail software defects or issues.
- Communication:
They serve as a way for testers to communicate problems to developers and other stakeholders.
- Reproducibility:
Good bug reports include clear steps to reproduce the issue, allowing developers to easily verify the bug.
- Context:
They should provide relevant context, such as the test case, environment, and expected vs. actual outcomes.
- Severity and Priority:
Assessing the severity and priority of the bug helps prioritize fixing efforts.
- Visual Evidence:
Screenshots, videos, or logs can be included to visually demonstrate the issue and aid in understanding.
- Bug Tracking:
Bug reporting is often integrated with bug tracking systems to manage, track, and monitor the status of reported bugs.

Elements of a Good Bug Report:

- Clear Title: A concise and descriptive title summarizing the issue.
- Detailed Description: A clear explanation of the issue, including when and how it occurred.
- Steps to Reproduce: Step-by-step instructions on how to recreate the bug.
- Expected vs. Actual Results: What was expected to happen versus what actually happened.
- Environment Details: Information about the environment where the bug was found (e.g., browser, operating system).

- **Visual Proof:** Screenshots, videos, or logs that demonstrate the bug.
- **Severity and Priority:** An assessment of the bug's impact and urgency.
- **Additional Information:** Relevant console logs, network requests, or source URLs can be included.

Benefits of Effective Bug Reporting:

- **Improved Software Quality:**
By identifying and addressing bugs early, the overall quality of the software is enhanced.
- **Faster Debugging:**
Well-written bug reports allow developers to quickly understand and fix issues.
- **Streamlined Development Process:**
Effective bug reporting facilitates better communication and collaboration between testers and developers.
- **Enhanced User Experience:**
By fixing bugs, the user experience is improved, leading to increased user satisfaction.

BUG LIFE CYCLE IN SOFTWARE DEVELOPMENT

During the development of any software product the development teams follow the Software Development Life Cycle (SDLC) processes. But the development process is not so easy and always runs smoothly. During the development process when a product is being developed different types of defects or bugs arise with the product. So these defects are identified and resolved throughout the development process just to deliver a good quality software product at last.

WHAT IS A BUG/DEFECT?

A defect is an error or bug in an application that is created during the building or designing of software due to which software starts to show abnormal behaviors during its use. So it is one of the important responsibilities of the tester to find as many as defects possible to ensure the quality of the product is not affected and the end product is fulfilling all requirements perfectly for which it has been designed and provides the required services to the end-user. Because as much as defects will be identified and resolved then the software will behave perfectly as per expectation.

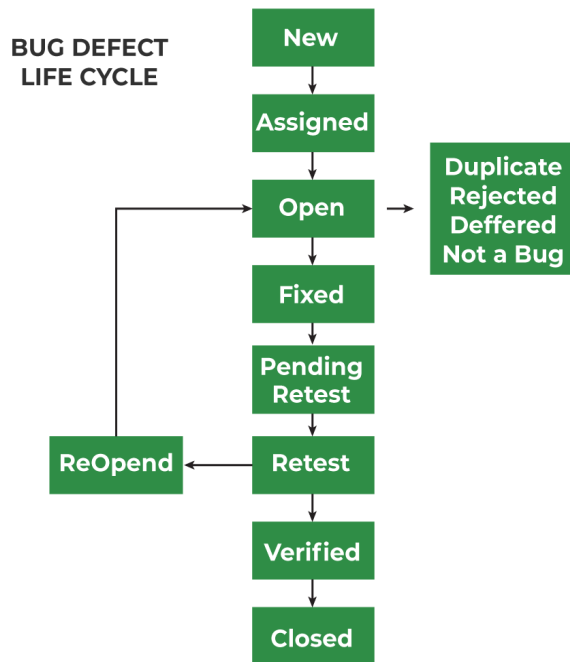
What is the Defect Life Cycle?

In the Software Development Process, the Defect Life Cycle is the life cycle of a defect or bug that it goes through covering a specific set of states in its entire life. Mainly bug life cycle refers to its entire state starting from a new defect detected to the closing off of that defect by the tester. Alternatively, it is also called a Bug Life Cycle.

- The journey of the Defect Cycle varies from organization to organization and also from project to project because development procedures and platforms as well as testing methods and testing tools differ depending upon organizations and projects.
- The number of states that a defect goes through also varies depending upon the different tools used and processes followed during the software testing.
- The objective of the defect lifecycle is to easily coordinate and communicate the current status of the defect and thus help to make the defect-fixing process efficient.
- □

DEFECT STATUS

Defect status or Bug status is the current state from which the defect is currently going through. The number of states the defect goes through varies from project to project.



1. New: When any new defect is identified by the tester, it falls in the 'New' state. It is the first state of the Bug Life Cycle. The tester provides a proper Defect document to the Development team so that the development team can refer to Defect Document and can fix the bug accordingly.

2. Assigned: Defects that are in the status of 'New' will be approved and that newly identified defect is assigned to the development team for working on the defect and to resolve that. When the defect is assigned to the developer team the status of the bug changes to the 'Assigned' state.

3. Open: In this 'Open' state the defect is being addressed by the developer team and the developer team works on the defect for fixing the bug. Based on some specific reason if the developer team feels that the defect is not appropriate then it is transferred to either the 'Rejected' or 'Deferred' state.

4. Fixed: After necessary changes of codes or after fixing identified bug developer team marks the state as 'Fixed'.

5. Pending Retest: During the fixing of the defect is completed, the developer team passes the new code to the testing team for retesting. And the code/application is pending for retesting on the Tester side so the status is assigned as 'Pending Retest'.

6. Retest: At this stage, the tester starts work of retesting the defect to check whether the defect is fixed by the developer or not, and the status is marked as 'Retesting'.

7. Reopen: After 'Retesting' if the tester team found that the bug continues like previously even after the developer team has fixed the bug, then the status of the bug is again changed to 'Reopened'. Once again bug goes to the 'Open' state and goes through the life cycle again. This means it goes for Re-fixing by the developer team.

8. Verified: The tester re-tests the bug after it got fixed by the developer team and if the tester does not find any kind of defect/bug then the bug is fixed and the status assigned is 'Verified'.

9. Closed: It is the final state of the Defect Cycle, after fixing the defect by the developer team when testing found that the bug has been resolved and it does not persist then they mark the defect as a 'Closed' state.